

Behind the Scenes of LLM Pre-training: StarCoder Use Case

Loubna Ben Allal
Hugging Face



Stanford
University

Today's plan

What does it take to train a good LLM?

No moat...



Google "We Have No Moat, And Neither Does OpenAI"

Leaked Internal Google Document Claims Open Source AI Will Outcompete Google and OpenAI



DYLAN PATEL AND AFZAL AHMAD

MAY 04, 2023 • PAID



687



10

Share

The text below is a very recent leaked document, which was shared by an anonymous individual on a public Discord server who has granted permission for its republication. It originates from a researcher within Google. We have verified its authenticity. The only modifications are formatting and removing links to internal web pages. The document is only the opinion of a Google employee, not the entire firm. We do not agree with what is written below, nor do other researchers we asked, but we will publish our opinions on this in a separate piece for subscribers. We simply are a vessel to share this document which raises some very interesting points.

2024: GPT4 performance on **your device**

Rank* (UB)	Model	Arena Elo	95% CI
1	GPT-4-Turbo-2024-04-09	1259	+4/-3
2	GPT-4-1106-preview	1253	+2/-3
2	Claude 3 Opus	1251	+3/-3
2	Gemini 1.5 Pro API-0409-Preview	1250	+3/-3
2	GPT-4-0125-preview	1247	+3/-2
6	Llama-3-70b-Instruct	1210	+3/-4
6	Bard (Gemini Pro)	1209	+5/-6
7	Claude 3 Sonnet	1201	+2/-3

Runs on a (large) **consumer desktop**

<https://huggingface.co/spaces/lmsys/chatbot-arena-leaderboard>, 2024/05/06

The rise of open models

 Data Center Dynamics

Meta to operate "600000 H100 GPU equivalents of compute" by year-end

Meta expects to field a fleet of 600,000 GPUs by the end of 2024. CEO Mark Zuckerberg told The Verge that the number includes some 340,000...

18 Jan 2024

****GPT-4 used 25'000 A100s for 3-4 months***

 Synced

DeepMind's Gemma: Advancing AI Safety and Performance with Open Models

Large Language Models (LLMs) have proven their mettle across a spectrum of real-world applications, ranging from language modeling to visual...



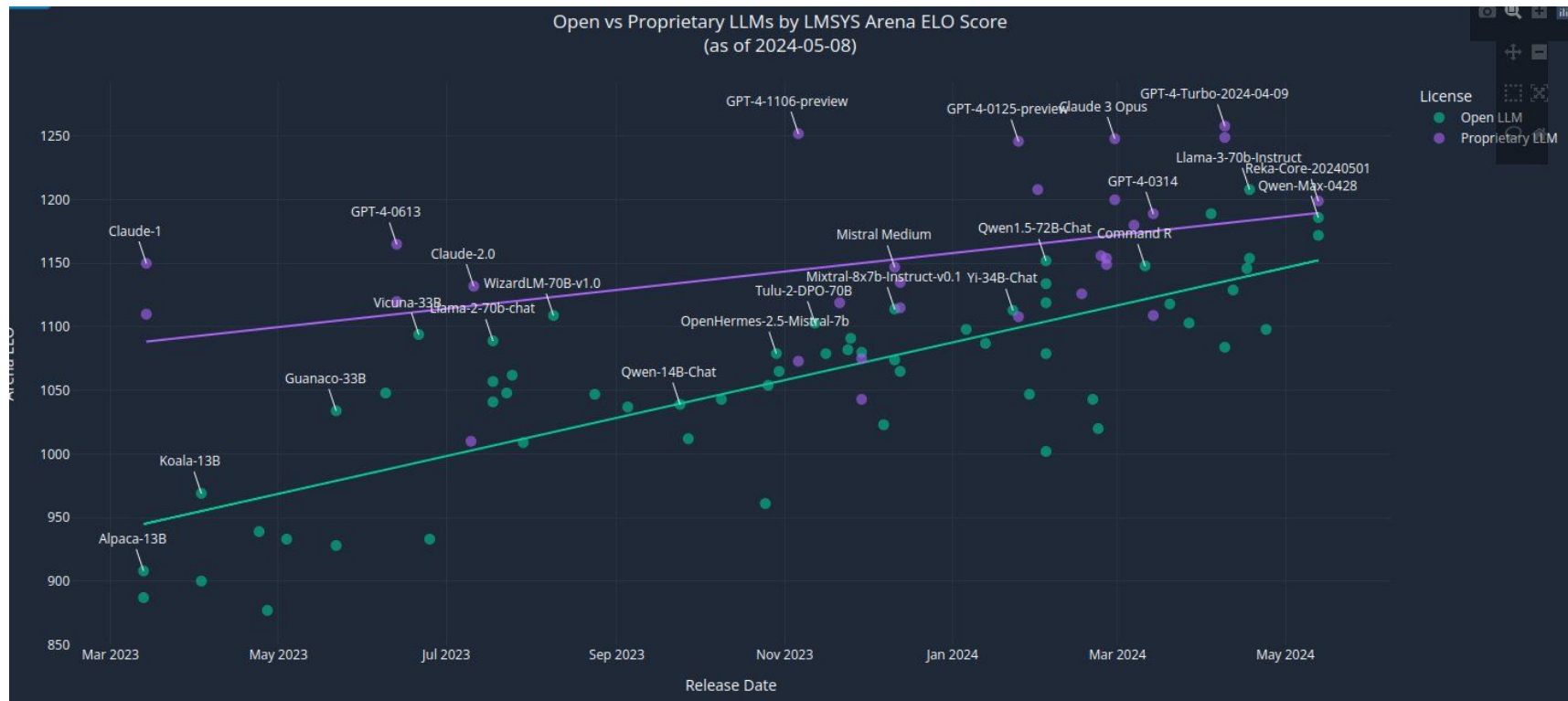
Mistral AI, a Paris-based OpenAI rival, closed its \$415 million funding round

Romain Dillet @romaindillet / 12:47 PM GMT+1 • December 11, 2023

 Comment



The rise of open models



Limitations

Missing details about data and model training to:

- Avoid legal scrutiny
- Maintain competitive edge

What do we need to train good LLMs?





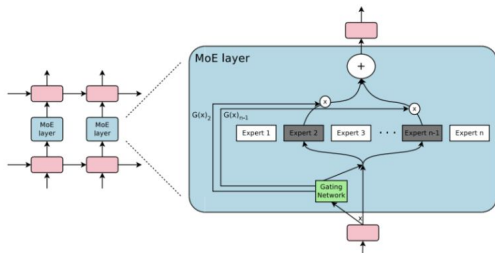
What do we need to train good LLMs?

Model



GPUs

Data





What do we need to train good LLMs?

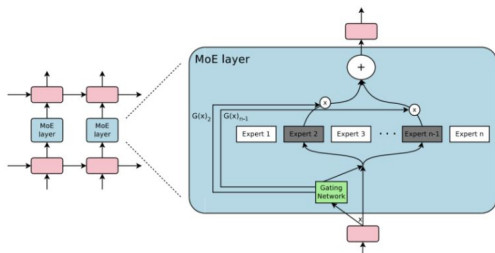
Model



GPUs

Ask Jensen

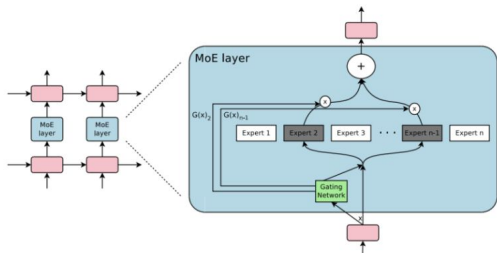
Data





What do we need to train good LLMs?

Model



GPUs

Ask Jensen



Data

The backbone of good
LLMs!

How to get good data for LLM
training?

Training data of LLMs

- **How much data** for which model size?
- Where to **get the data**?
- How to **filter it**?

Training data of LLMs: **How much data?**

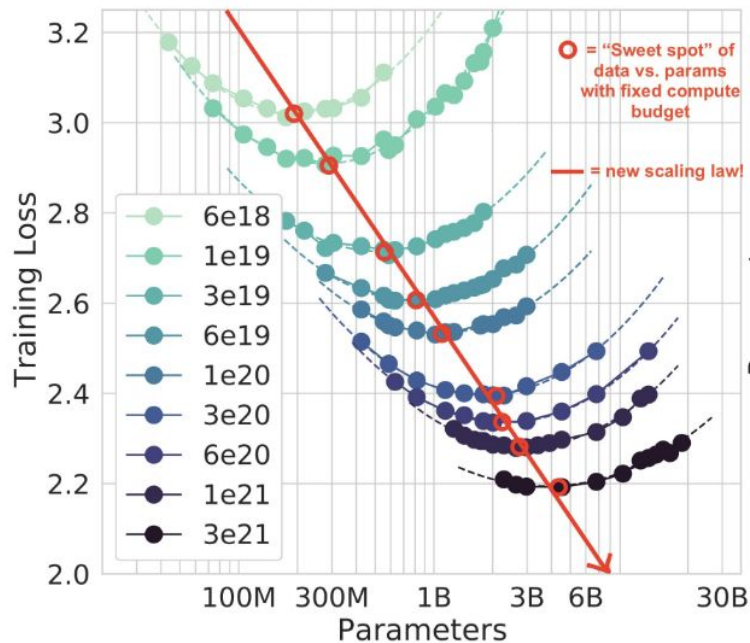
Answer: scaling laws

How to allocate a compute budget between
data size and model size?

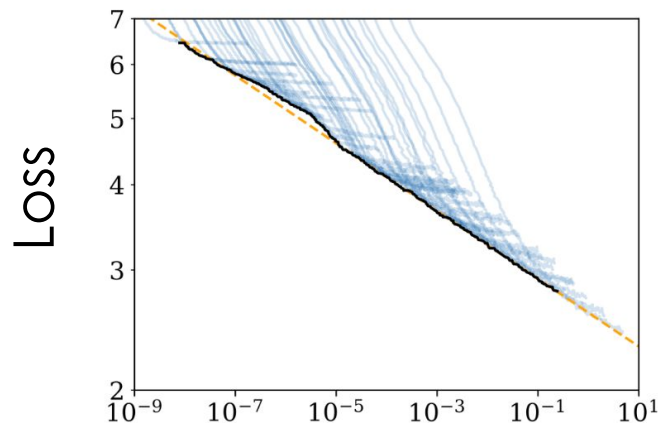
Scaling laws

- Kaplan et al. (2020) <https://arxiv.org/abs/2001.08361>
 - **10x** increase in compute => **5.5x** increase in parameter count and a **1.8x** increase in training tokens
 - GPT3 175B trained on ~300B tokens
- Chinchilla (2022) <https://arxiv.org/abs/2203.15556>
 - Cosine scheduler error in Kaplan et al.
 - Scale data and model size **equally**
 - Chinchilla 65B trained on ~1.6T tokens

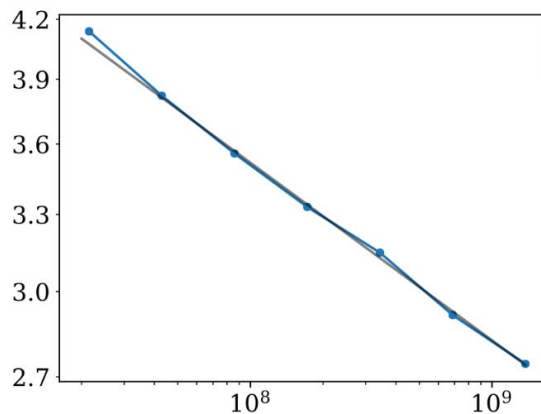
Scaling laws



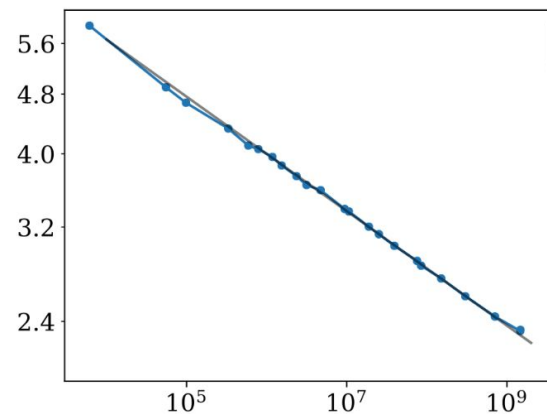
Scaling laws



Compute



Data



Model size

Scaling laws

Chinchilla scaling laws
model 3.2x, data 3.2x

Chinchilla
65B params, 1.6T tokens

Compute-optimal
≠ optimal

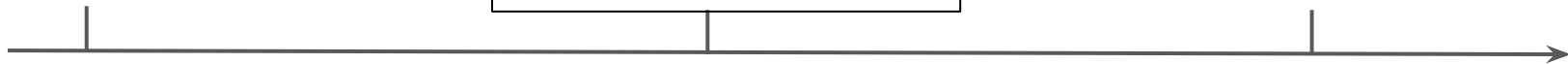
LLaMA
7B params, 1T tokens

LLaMA-3
8B params, 15T tokens

March 2022

February 2023

April 2024



From GPT 1 → 4

	Dataset size (Billion tokens)	Model size (Billion parameter)	
GPT 1:	1-2	0.11	
GPT 2:	10-20	1.4	100x
GPT 3:	300	175	2000x
GPT 4:	10'000	1'800	300x

↪ **GPT-4 train cost: ~\$100M**

Inference gets more **expensive**

	Model size (Billion parameter)	GPU memory (16bit, GB)	
GPT 1:	0.11	0.22	
GPT 2:	1.4	2.8	10x
GPT 3:	175	350	100x
GPT 4:	1'800	3600	10x

↪ Time per token also increases with model size!

Scaling laws: **Compute-optimal $\neq$ optimal**

- The **inference cost** isn't considered in the scaling laws
- **Smaller models trained longer** are more cost effective
- Now we choose to train smaller models past Chinchilla point and pay a **compute overhead** during training

Scaling laws: **Compute-optimal $\neq$ optimal**

“Harm’s law”: Compute overhead for training past Chinchilla

Training configuration

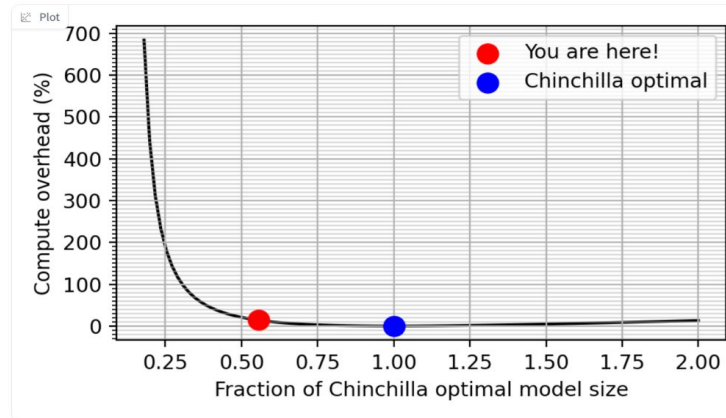
Model size (in B parameters):

7

Dataset size (in B tokens):

1000

Harm's law



Training summary

Training compute	Training cost	Training time	Total GPU hours
4.20E+22 TFLOPs	\$0.07M	0.98 days	0.02M

Chinchilla and Training/Inference Trade-off

Optimal model/dataset size for training compute and how it translates to training overhead and inference savings according to Harm's law

Chinchilla optimal model	Chinchilla optimal dataset	Training overhead	Inference savings
12.60B parameters	555.52B tokens	13.92%	44.45%

<https://www.harmdevries.com/post/model-size-vs-compute-overhead/>

<https://huggingface.co/spaces/lvwerra/train-llm>

Scaling laws: further reading

- Scaling Data-Constrained Language Models: <https://arxiv.org/abs/2305.16264>
 - possibility of training up to **4 epochs**
 - e.g The Stack v2 exhausts all publicly available code
- DeepSeek LLM: Scaling Open-Source Language Models with Longtermism
<https://arxiv.org/abs/2401.02954>
 - **Data quality** influences scaling behavior
 - For higher quality datasets , more compute should be allocated to model size

How much data to train LLMs?



Where to **find** the data?



LLMs training data

- **Web data**
- **GitHub code**
- Curated sources:
 - Wikipedia
 - Arxiv
 - StackExchange
- Synthetic data

LLMs training data: **web data**

- Process CommonCrawl:
 - public repository of crawled web pages
 - requires heavy filtering at a very large scale (424.7 TiB of content in the latest dump, 95 dumps)
- Use existing filtered web datasets

LLMs training data: web data

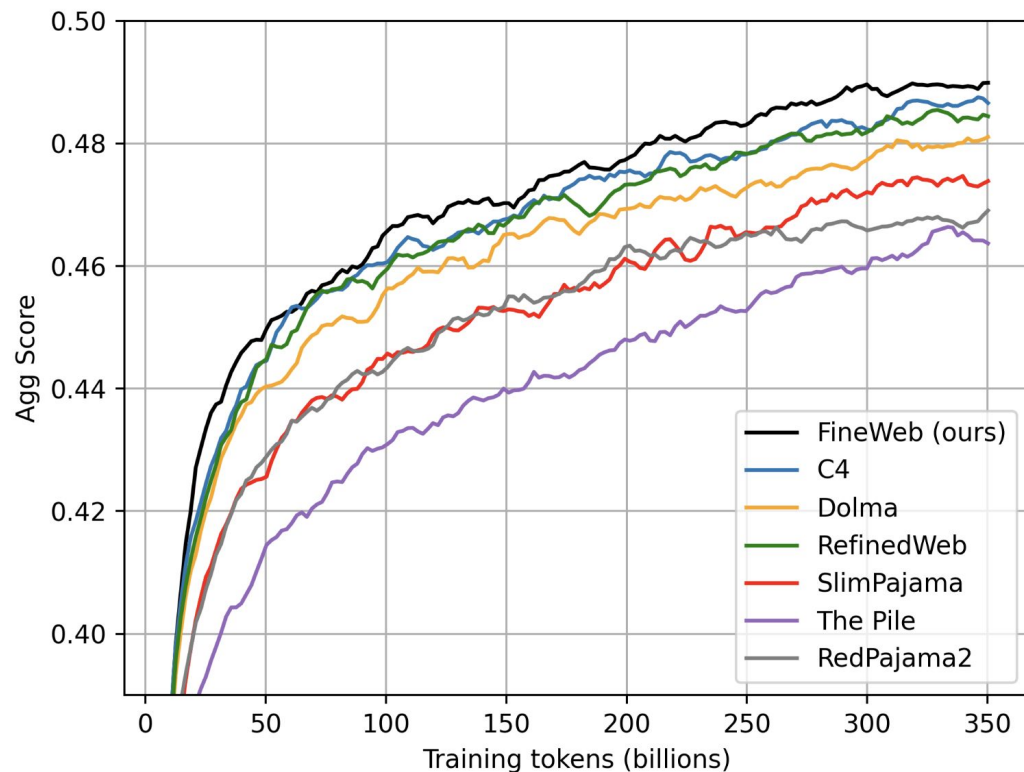
F^{ine}ne^uweb

The finest collection of data the web has to offer

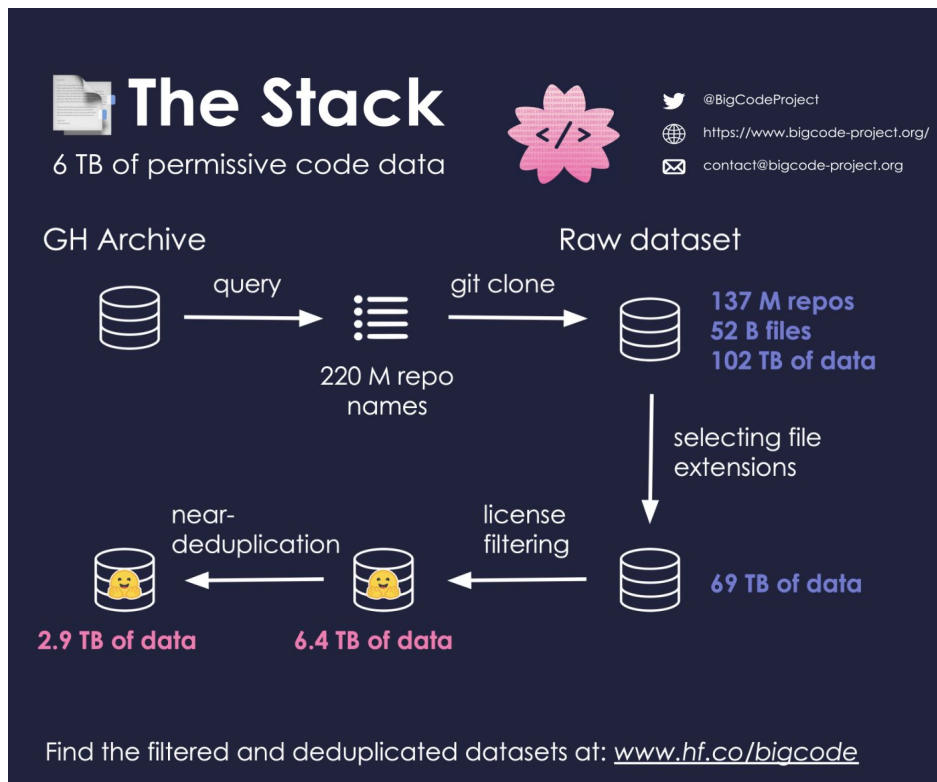


"15 trillion tokens of the finest data the  web has to offer"

LLMs training data: web data

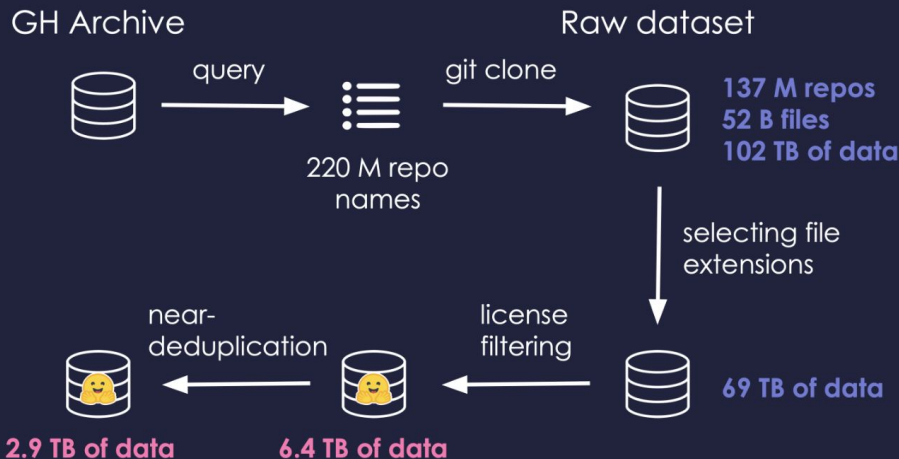


LLMs training data: **code data**



The Stack v1

Data collection



Find the filtered and deduplicated datasets at: www.hf.co/bigcode

Data inspection + Opt-out

 **BigCode**

The Stack is an open governance interface between the AI community and the open source community.

Am I in The Stack?

As part of the BigCode project, we released and maintain [The Stack](#), a 3.1 TB dataset of permissively licensed source code in 30 programming languages. One of our goals in this project is to give people agency over their source code by letting them decide whether or not it should be used to develop and evaluate machine learning models, as we acknowledge that not all developers may wish to have their data used for that purpose.

This tool lets you check if a repository under a given username is part of The Stack dataset. Would you like to have your data removed from future versions of The Stack? You can opt-out following the instructions [here](#).

The Stack version:

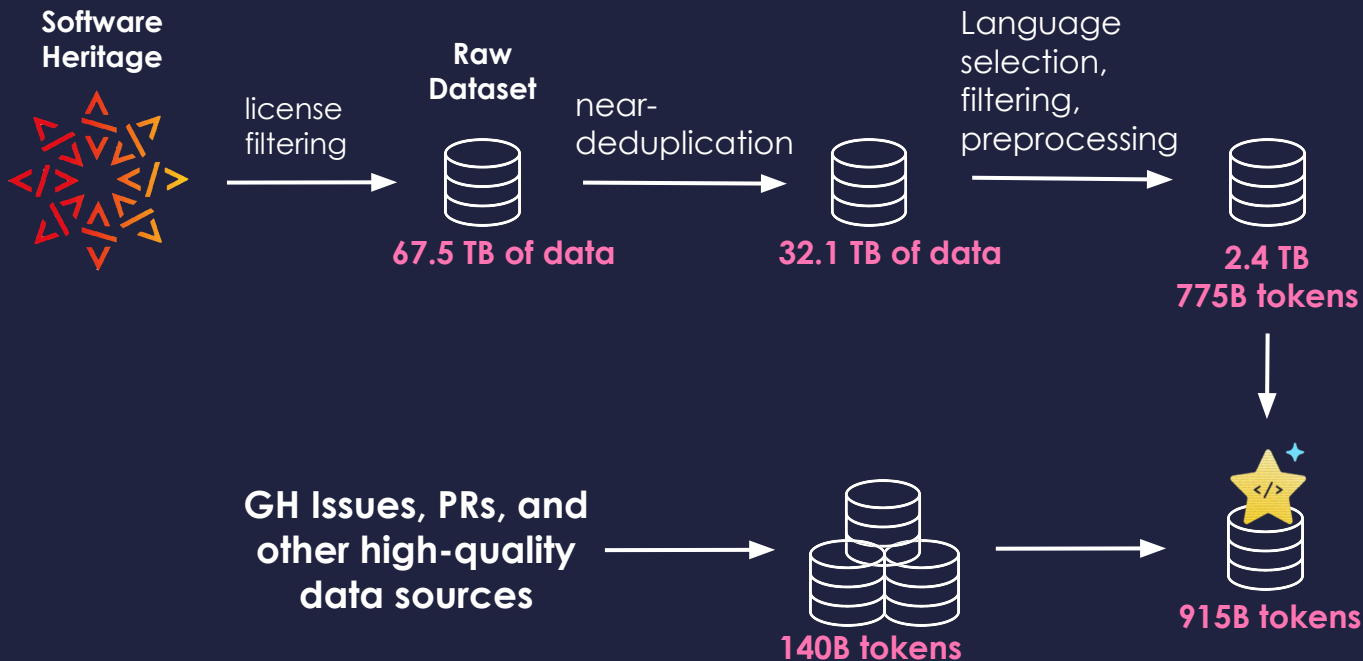
v1.1

Your GitHub username:

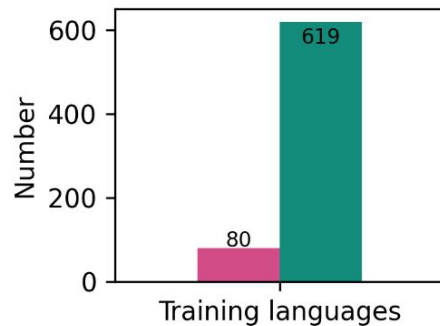
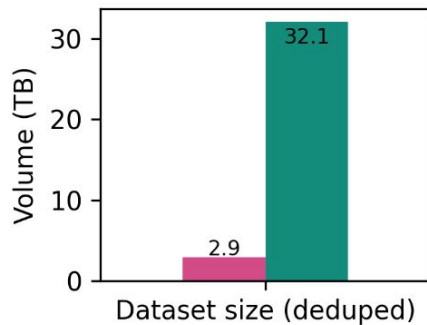
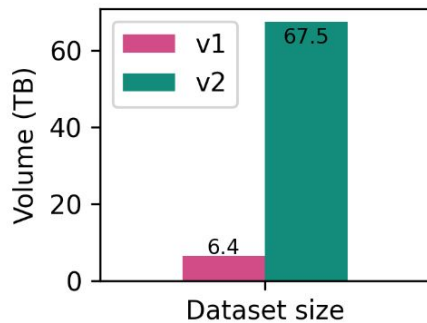
Check!

The Stack v2

Data collection



The Stack v2



LLMs training data: **synthetic data**

Textbooks Are All You Need

Suriya Gunasekar, Yi Zhang, Jyoti Aneja, Caio César Teodoro Mendes, Allie Del Giorno, Sivakanth Gopi, Mojan Javaheripi, Piero Kauffmann, Gustavo de Rosa, Olli Saarikivi, Adil Salim, Shital Shah, Harkirat Singh Behl, Xin Wang, Sébastien Bubeck, Ronen Eldan, Adam Tauman Kalai, Yin Tat Lee, Yuanzhi Li

We introduce phi-1, a new large language model for code, with significantly smaller size than competing models: phi-1 is a Transformer-based model with 1.3B parameters, trained for 4 days on 8 A100s, using a selection of “textbook quality” data from the web (6B tokens) and synthetically generated textbooks and exercises with GPT-3.5 (1B tokens). Despite this small scale, phi-1 attains pass@1 accuracy 50.6% on HumanEval and 55.5% on MBPP. It also displays surprising emergent properties compared to phi-1-base, our model before our finetuning stage on a dataset of coding exercises, and phi-1-small, a smaller model with 350M parameters trained with the same pipeline as phi-1 that still achieves 45% on HumanEval.

2.5 Training Data

Claude 3 models are trained on a proprietary mix of publicly available information on the Internet as of August 2023, as well as non-public data from third parties, data provided by data labeling services and paid contractors, and **data we generate internally**. We employ several data cleaning and filtering methods, including deduplication and classification. The Claude 3 suite of models have not been trained on any user prompt or output data submitted to us by users or customers, including free users, Claude Pro users, and API customers.

- We found that previous generations of Llama are good at identifying high-quality data, so we used Llama 2 to help build the text-quality classifiers that are powering Llama 3. **We also leveraged synthetic data to train in areas such as coding, reasoning, and long context**. For example, we used synthetic data to create longer documents to train on.

LLMs training data: **synthetic data**

Cosmopedia is a dataset of synthetic textbooks, blogposts, stories, posts and WikiHow articles generated by Mixtral-8x7B-Instruct-v0.1. The dataset contains over **30 million files** and **25 billion tokens**, making it the largest open synthetic dataset to date.



LLMs training data: **synthetic data**

Cosmopedia is a dataset of synthetic textbooks, blogposts, stories, posts and WikiHow articles generated by Mixtral-8x7B-Instruct-v0.1. The dataset contains over **30 million files** and **25 billion tokens**, making it the largest open synthetic dataset to date.

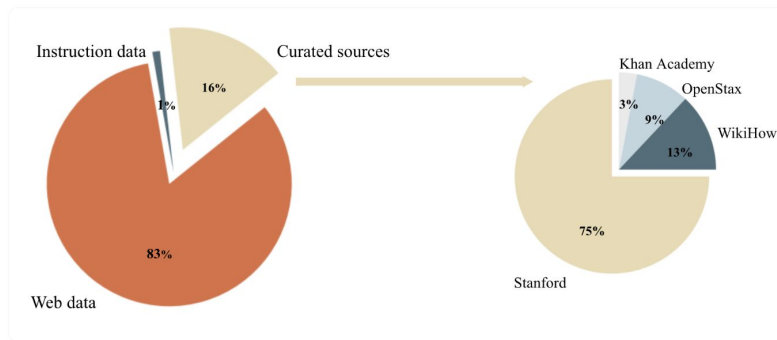


Figure 2. The distribution of data sources for building Cosmopedia prompts (left plot) and the distribution of sources inside the Curated sources category (right plot).

Where to **find** the data?



How to **filter** the data?



LLMs training data filtering

I – Training

2 Pretraining

Our approach to pretraining is to train a standard dense transformer architecture on a heavily engineered large pretraining corpora, where our underlying assumption is that when trained on extensive data of high-enough quality, a standard architecture can exhibit advanced capability. This is to say, we may not need much architectural modification, although we have indeed conducted extensive preliminary architectural experiments. In the following subsections, we first detail our data engineering pipeline, then briefly discuss the model architecture.

Yi: Open Foundation Models by 01.AI

<https://arxiv.org/abs/2403.04652>



A little guide to building Large Language Models in 2024

<https://www.youtube.com/watch?v=2-SPH9hIKT8>

LLMs training data filtering

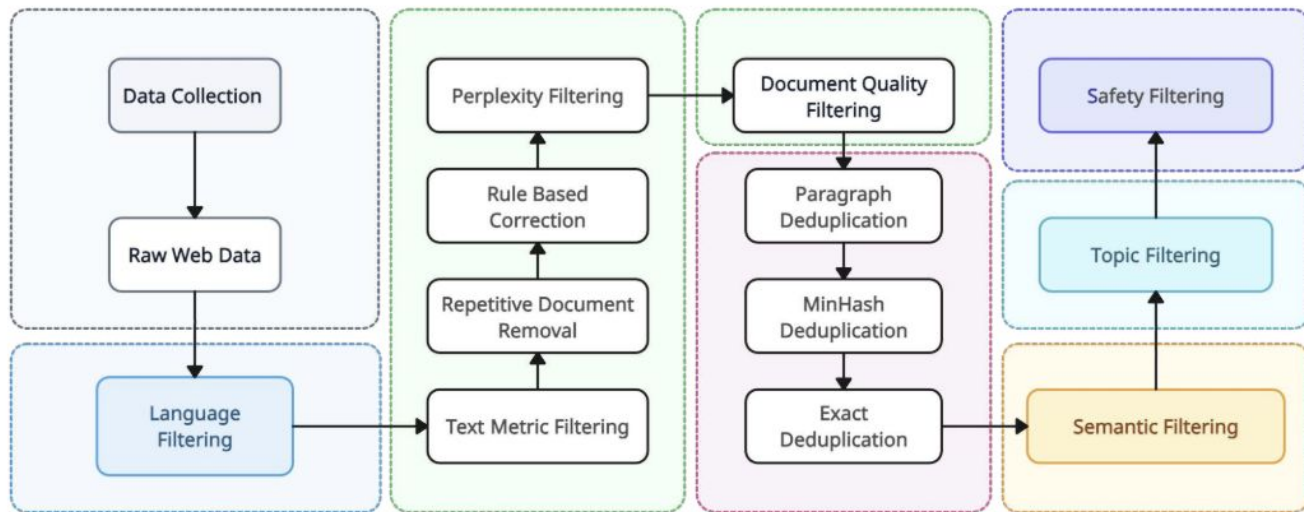
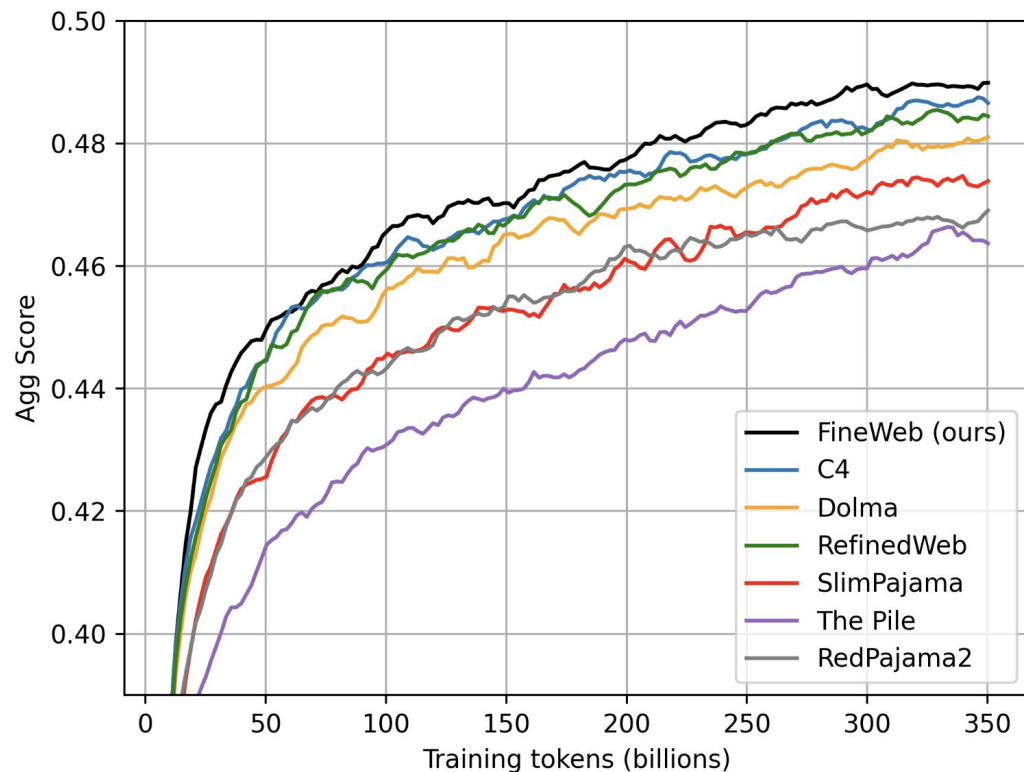


Figure 1: Yi's pretraining data cleaning pipeline.

Yi: Open Foundation Models by 01.AI

<https://arxiv.org/abs/2403.04652>

LLMs training data: FineWeb



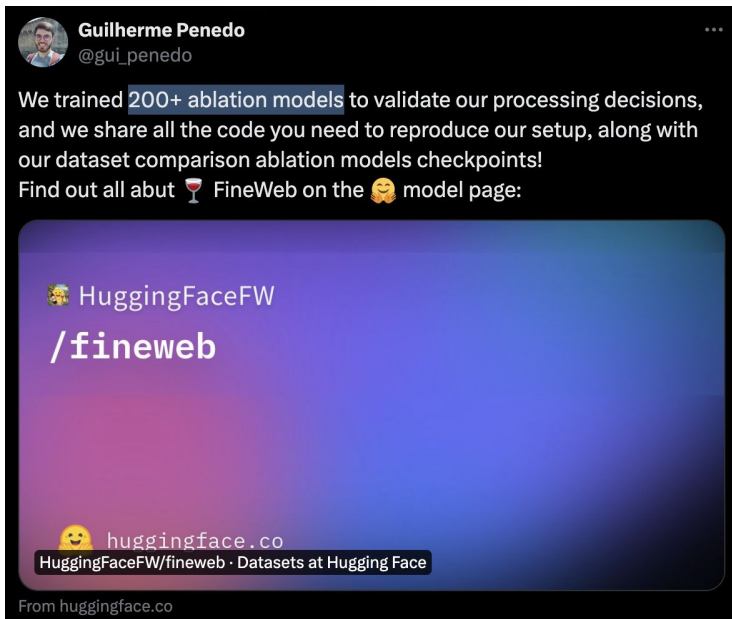
LLMs training data filtering

How to find the best **filtering techniques**?

- Manual inspection
- Train ablation models (small model on subsets of the data):
 - Evaluate on High Signal benchmarks (e.g Hellaswag, MMLU...)
 - Train with different seeds to reduce noise

LLMs training data filtering

- **200+** ablation models for FineWeb filtering



StarCoder use case: filtering The Stack v1

6.4TB of source code in 358 programming languages



800 GB of code in 86 programming languages, with GitHub Issues, Jupyter Notebooks and Git Commits

StarCoder use case: filtering The Stack v2

32.1TB of source code in 600+ programming languages



3.6TB of code in 600+ programming languages, with GitHub Issues, Pull Requests, Jupyter & Kaggle Notebooks and High quality math and code datasets

StarCoder use case: filtering The Stack v1

- **Language selection**

- 86 languages out of 358 (popular languages, excluding configs)
- Includes GitHub issues, git commits, structured Jupyter notebooks

- **Data quality inspection**

- Inspect 100 samples per source and derive filtering heuristics

StarCoder use case: filtering The Stack v1

near-deduplication: MinHash + LSH

- language agnostic and shows consistent performance gains

Dataset	Filtering	pass@1	pass@10	pass@100
Codex (300M)	unknown	13.17	20.17	36.27
CodeGen (350M)	unknown	12.76	23.11	35.19
Python all-license	None	13.11	21.77	36.67
	Near-dedup	17.34	27.64	45.52
Python permissive-license	None	10.99	15.94	27.21
	Near-dedup	12.89	22.26	36.01

StarCoder use case: filtering The Stack v1

- **Removing PII**

- StarPII: Model to detect/remove names, emails, password and keys

- **Data Decontamination**

- Decontaminate from evaluation benchmarks

StarCoder use case: filtering The Stack

Data formatting

- Code for StarCoder1

`<reponame>reponame<filename>filepath<gh_stars>stars\ncode<eos>`

- Code for StarCoder2 (repository aware)

`<repo_name>reponame<file_sep>filepath1\ncode1<file_sep>filepath2\ncode2... <|endoftext|>`

- GitHub Issues

`<issue_start>title + USERID: comment<issue_comment>USERID: Comment
... <issue_closed (optional)> <eos>`

StarCoder use case: filtering The Stack

Data formatting

- Git Commits & Jupyter Notebooks



```
- Model input:  
<commit_before> def fibonacci(n):  
<commit_msg> add type hints to function  
<commit_after> def
```

```
- Model output:  
fibonacci(n: int) -> list[int]:
```



```
- Model input:  
<jupyter_code>numbers = [1, 9, 8, 3, 27]  
print([n*2 for n in numbers])<jupyter_output>
```

```
- Model output:  
[2, 18, 16, 6, 54]
```

StarCoder use case: filtering The Stack v2

StarCoder2 training data

Dataset	Dedup	Malicious Code	Decontaminate	Opt-out	PII
Source Code	Yes	Yes	Yes	Yes	StarPII
Pull Requests	Yes	Yes	Yes	Yes	StarPII + Usernames
Jupyter/Kaggle Notebooks	Yes	Yes	Yes	Yes/No	StarPII
Issues	Yes	Yes	Yes	Yes	StarPII + Usernames
Docs	Yes	No	No	No	StarPII: Names, Keys, Emails
LHQ	No	No	No	No	No
Arxiv	No	No	No	No	Email
OpenWebMath	No	No	Yes	No	StarPII: Names, Keys, Emails
Wikipedia	No	No	No	No	No
StackExchange	No	No	Yes	No	StarPII + Usernames

LLMs training data filtering

Frameworks for data filtering and training:

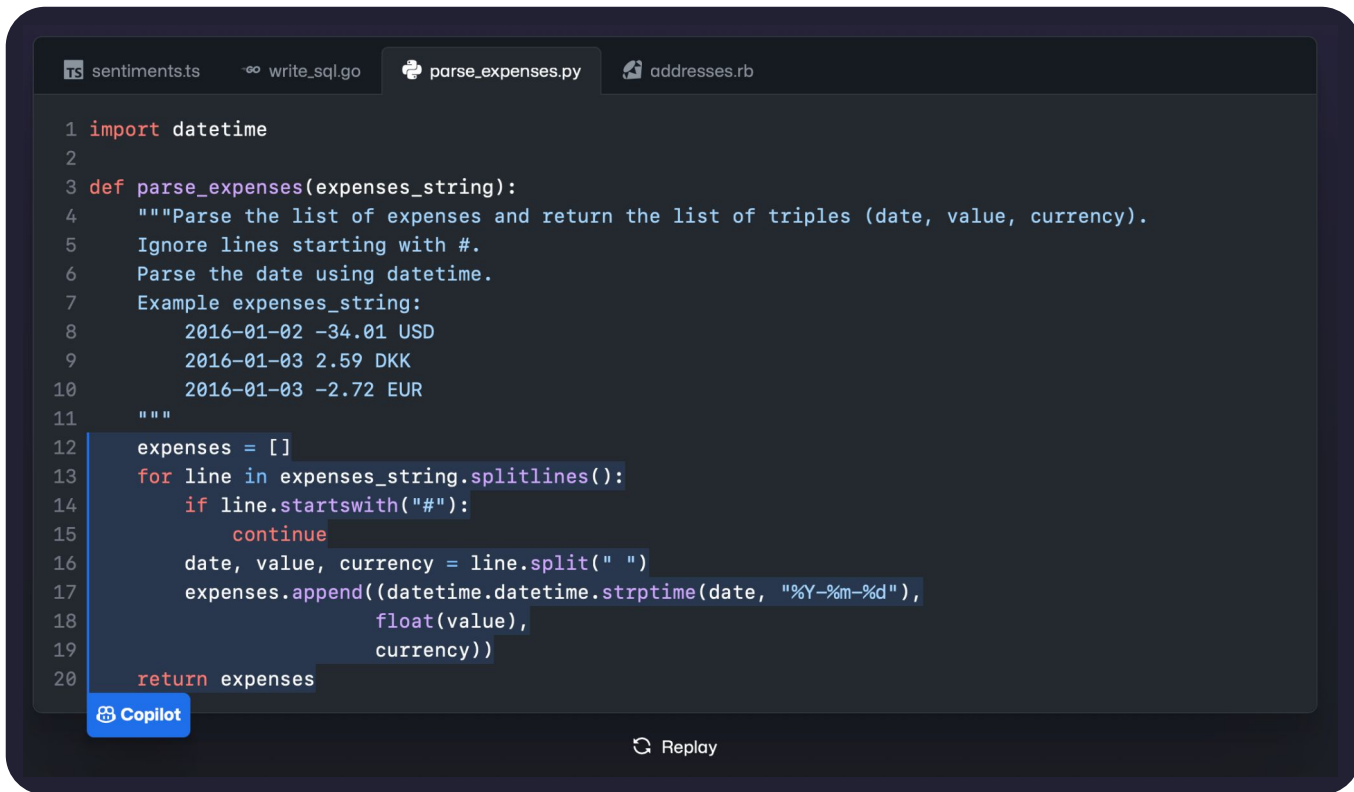
- **datasets** library
- **datatrove**: library for filtering, deduplicating and tokenizing text data at **large scale**
<https://github.com/huggingface/datatrove>
- Integrated with **nanotron** framework for 3B parallelism training
<https://github.com/huggingface/nanotron/> and **lighteval**
<https://github.com/huggingface/lighteval>
- Bigcode processing code: <https://github.com/bigcode-project/bigcode-dataset>

How to **filter** the data?



More about LLMs for code

How it started: GitHub Copilot in 2021



```
ts sentiments.ts  ∞ write_sql.go  parse_expenses.py  addresses.rb

1 import datetime
2
3 def parse_expenses(expenses_string):
4     """Parse the list of expenses and return the list of triples (date, value, currency).
5     Ignore lines starting with #.
6     Parse the date using datetime.
7     Example expenses_string:
8         2016-01-02 -34.01 USD
9         2016-01-03 2.59 DKK
10        2016-01-03 -2.72 EUR
11    """
12    expenses = []
13    for line in expenses_string.splitlines():
14        if line.startswith("#"):
15            continue
16        date, value, currency = line.split(" ")
17        expenses.append((datetime.datetime.strptime(date, "%Y-%m-%d"),
18                        float(value),
19                        currency))
20    return expenses
```

 Copilot

 Replay

How it's going: Over 1.7k open models trained on code

 **Hugging Face**

Search models, datasets, users...

Models Datasets Spaces Posts Docs Solutions Pricing



Tasks Libraries Datasets **Languages** Licenses Other

code

code x

Reset Languages

Models 1,738

Filter by name

new Full-text search

Sort: Trending

 **stabilityai/stable-code-instruct-3b**
Text Generation • Updated 2 days ago • 2.21k • 95

 **microsoft/phi-2**
Text Generation • Updated Feb 6 • 740k • 3k

 **PipableAI/pip-library-etl-1.3b**
Text Generation • Updated 1 day ago • 830 • 19

 **microsoft/phi-1_5**
Text Generation • Updated Feb 6 • 81.2k • 1.25k

 **m-a-p/OpenCodeInterpreter-DS-33B**
Text Generation • Updated 27 days ago • 1.91k • 64

 **bartowski/stable-code-instruct-3b-GGUF**
Text Generation • Updated 5 days ago • 7

 **VAIBHAV22334455/JARVIS**
Text Generation • Updated 1 day ago • 164 • 6

 **stabilityai/stable-code-3b**
Text Generation • Updated 11 days ago • 20.1k • 590

 **bigscience/bloom**
Text Generation • Updated Jul 28, 2023 • 11k • 4.5k

 **Locutusque/OpenCerebrum-1.0-7b-DPO**
Text Generation • Updated 2 days ago • 158 • 11

 **bigcode/starcoder**
Text Generation • Updated 9 days ago • 14.9k • 2.68k

 **bigcode/starcoder2-15b**
Text Generation • Updated 24 days ago • 128k • 471

 **aurora-m/aurora-m-biden-harris-redteamed**
Text Generation • Updated 2 days ago • 395 • 13

 **codellama/CodeLlama-7b-hf**
Text Generation • Updated Jan 29 • 152k • 275

Strong Instruction-tuned and **base models**

★ Big Code Models Leaderboard

Inspired from the 🤖 [Open LLM Leaderboard](#) and 🤖 [Open LLM-Perf Leaderboard](#), we compare performance of base multilingual code generation models on [HumanEval](#) benchmark and [MultiPL-E](#). We also measure throughput and provide information about the models. We only compare open pre-trained multilingual code models, that people can start from as base models for their trainings.

🔍 Evaluation table

📊 Performance Plot

📄 About

🚀 Submit results

📄 See All Columns

🔍 Search for your model and press ENTER...

⚙️ Filter model types

☒ all

☐ base

☐ instruction-tuned

T	▲	Model	▲	Win Rate	▲	humaneval-python	▲	java	▲	javascript	▲	c++	▲
◆		DeepSeek-Coder-33b-instruct		47.17		80.02		52.03		65.13		62.36	
◆		DeepSeek-Coder-7b-instruct		45.92		80.22		53.34		65.8		59.66	
◆		OpenCodeInterpreter-DS-6.7B		45.42		73.2		51.41		63.85		60.01	
◆		Phind-Codellama-34B-v2		44.5		71.95		54.06		65.34		59.59	
◆		Phind-Codellama-34B-v1		43.42		65.85		49.47		64.45		57.81	
◆		Phind-Codellama-34B-Python-v1		41.88		70.22		48.72		66.24		55.34	
◆		Codellama-70b-Instruct		39.83		75.6		47.2		57.76		48.45	
◆		WizardCoder-Python-34B-V1.0		39.5		70.73		44.94		55.28		47.2	
●		Codellama-70b		39.33		52.44		44.72		56.52		49.69	
●		DeepSeek-Coder-33b-base		39.33		52.45		43.77		51.28		51.22	
●		Codellama-70b-Python		38.75		55.49		45.96		56.52		49.69	
●		StarCoder2-15B		37		44.15		33.86		44.24		41.44	
●		DeepSeek-Coder-7b-base		35.5		45.83		37.72		45.9		45.53	

The Landscape of open code LLMs



- CodeLlama
- CodeLlama-Instruct
- 7B, 13B, 70B



BigCode

- The Stack dataset
- StarCoder & StarCoder2
- 3B, 7B, 15B sizes
- StarChat2 (with H4 team)



deepseek coder

- DeepSeek-Coder
- DeepSeek-Coder-Instruct
- 1B, 7B, 33B

Others: Granite from IBM, CodeQwen from Qwen team, CodeGen from SalesForce, StableCode from StabilityAI...

BigCode: open-scientific collaboration

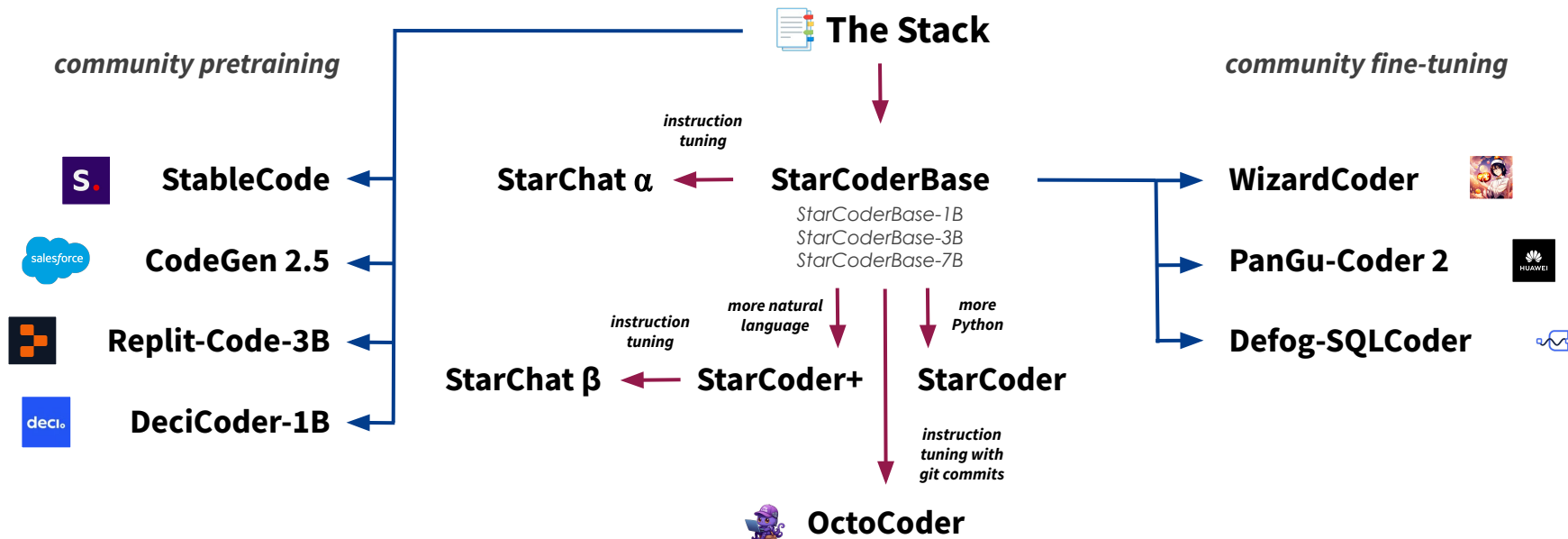
We are building LLMs for code in a collaborative way:

- Full **data transparency**
- Open source processing and training **code**
- **Model weights** released with commercial friendly license

1100+ researchers, engineers, lawyers, and policy makers



BigCode Ecosystem



Open & Responsible Research on LLMs

Open-access datasets

[Data inspection](#)

[Opt-out available](#)

[PII removal](#)

[Attribution](#)

Open-access models

[Model weights available](#)

[Fine-tuning scripts](#)

[Low-precision inference](#)

Reproducible research

[Data preprocessing scripts](#)

[Model training framework](#)

[R&D notebooks](#)

[Evaluation Harness](#)

Documentation

[Dataset cards](#)

[Model cards](#)

[Governance card](#)

[Intellectual property](#)

[Code of conduct](#)

[OpenRAIL licenses](#)

From SantaCoder to StarCoder2 🚀



SantaCoder
Dec 2022

- 1.1B code generation model
- 3 languages
- **18%** Python score
- **Transparent dataset**
- **Open Access**



StarCoder
May 2023

- 15B code generation model
- 80+ languages
- **33%** Python score
- **Transparent dataset**
- **Open Access**



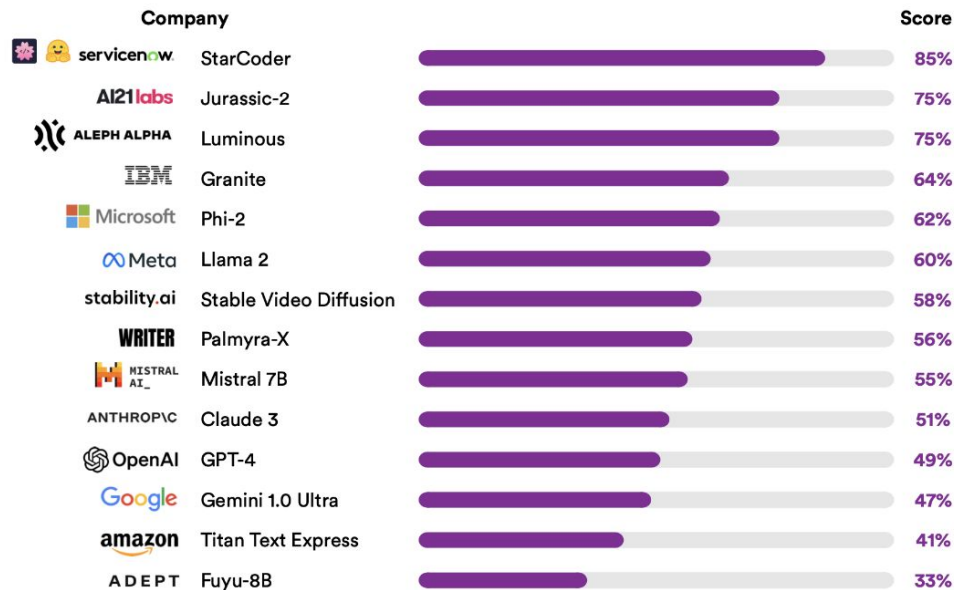
StarCoder2
Feb 2024

- 15B code generation model
- 600+ languages
- **46%** Python score
- **Transparent dataset**
- **Open Access**

Stanford Foundation Model Transparency Index

Foundation Model Transparency Index Total Scores, May 2024

Source: May 2024 Foundation Model Transparency Index





Evaluation

Search for your model and press ENTER...

Filter model types

☐ all

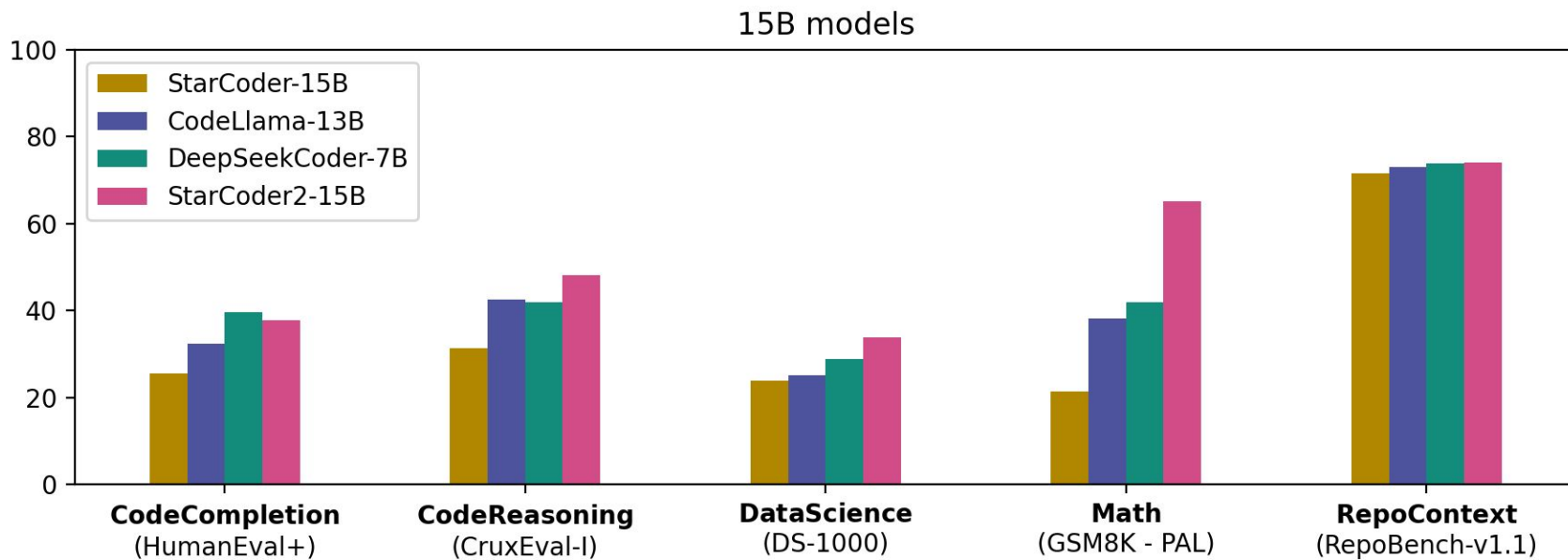
☒ base

☐ instruction-tuned

T	Model	Win Rate	humaneval-python	java	javascript	c++
●	CodeLlama-70b	39.33	52.44	44.72	56.52	49.69
●	DeepSeek-Coder-33b-base	39.33	52.45	43.77	51.28	51.22
●	CodeLlama-70b-Python	38.75	55.49	45.96	56.52	49.69
●	StarCoder2-15B	37	44.15	33.86	44.24	41.44
●	DeepSeek-Coder-7b-base	35.5	45.83	37.72	45.9	45.53
●	CodeLlama-34b	34.5	45.11	40.19	41.66	41.42
●	CodeLlama-34b-Python	33.88	53.29	39.46	44.72	39.09
●	CodeLlama-13b	29.96	35.07	32.23	38.26	35.81
●	CodeLlama-13b-Python	28.04	42.89	33.56	40.66	36.21
●	StarCoder2-7B	26.67	34.09	29.42	35.35	33.63
●	CodeLlama-7b	25	29.98	29.2	31.8	27.23
●	CodeLlama-7b-Python	23.38	40.48	29.15	36.34	30.34
●	StarCoder-15B	22.88	33.57	30.22	30.79	31.55

Source: <https://hf.co/spaces/bigcode/bigcode-models-leaderboard>

StarCoder2



Tooling

Auto-complete

```
Users > swayam > Desktop >  main.py > ...  
1 def is_prime(num):  
2   return False  
  
def is_prime(num):  
    if num == 2:  
        return True  
    if num % 2 == 0:  
        return False  
    for i in range(3, num, 2):  
        if num % i == 0:  
            return False
```

<https://marketplace.visualstudio.com/items?itemName=HuggingFace.huggingface-vscode>

Membership test

```
Users > swayam > Desktop >  main.py >  is_prime  
1 def is_prime(num):  
2   return False  
3  
4 def is_prime(num):  
5   if num == 2:  
6     return True  
7   if num % 2 == 0:  
8     return False  
9   for i in range(3, num, 2):  
10    if num % i == 0:  
11    return False
```

 Highlighted code was found in the stack. 

Source: HF Code Autocomplete (Extension) 

Customize Code Models: Code completion

Fine-tune an open code model on your codebase:

<https://huggingface.co/blog/personal-copilot>

Personal Copilot: Train Your Own Coding Assistant

Published October 27, 2023

[Update on GitHub](#)



[smangrul](#)
[Sourab Mangrulkar](#)



[sayakpaul](#)
[Sayak Paul](#)

In the ever-evolving landscape of programming and software development, the quest for efficiency and productivity has led to remarkable innovations. One such innovation is the emergence of code generation models such as [Codex](#), [StarCoder](#) and [Code Llama](#). These models have demonstrated remarkable capabilities in generating human-like code snippets, thereby showing immense potential as coding assistants.

However, while these pre-trained models can perform impressively across a range of tasks, there's an exciting possibility lying just beyond the horizon: the ability to tailor a code generation model to your specific needs. Think of personalized coding assistants which could be leveraged at an enterprise scale.

Leaderboards

The screenshot shows the 'Big Code Models Leaderboard' page on Hugging Face. The page header includes the Hugging Face logo, the space name 'bigcode/bigcode-models-leaderboard', and a 'Running' status. The main title is 'Big Code Models Leaderboard' with a star icon. Below the title, there is a paragraph explaining the leaderboard's purpose: comparing the performance of base multilingual code generation models on the HumanEval benchmark and MultiPL-E. The page features a navigation bar with 'Evaluation table', 'Performance Plot', 'About', and 'Submit results' buttons. A search bar is present with the placeholder text 'Search for your model and press ENTER...'. To the right of the search bar, there are filter buttons for 'all', 'base', 'instruction-tuned', and 'EXT external-evaluation'. The main content is a table listing various models and their performance metrics.

Spaces bigcode bigcode-models-leaderboard like 706 Running Logs

App Files Community 66 Settings

★ Big Code Models Leaderboard

Inspired from the 🤖 [Open LLM Leaderboard](#) and 🤖 [Open LLM-Perf Leaderboard](#), we compare performance of base multilingual code generation models on [HumanEval](#) benchmark and [MultiPL-E](#). We also measure throughput and provide information about the models. We only compare open pre-trained multilingual code models, that people can start from as base models for their trainings.

🔍 Evaluation table 📊 Performance Plot 📄 About 🚀 Submit results

🔍 See All Columns

🔍 Search for your model and press ENTER...

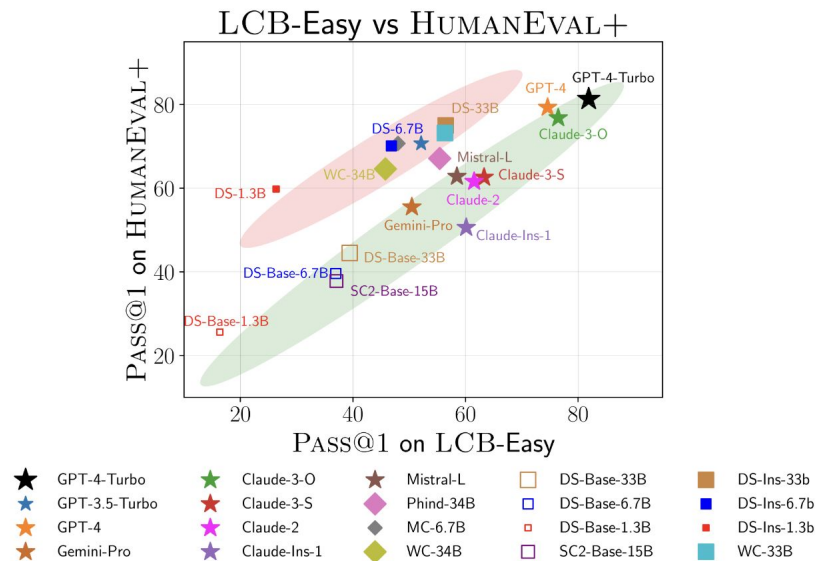
⚙️ Filter model types

☒ all ☐ base ☐ instruction-tuned ☐ EXT external-evaluation

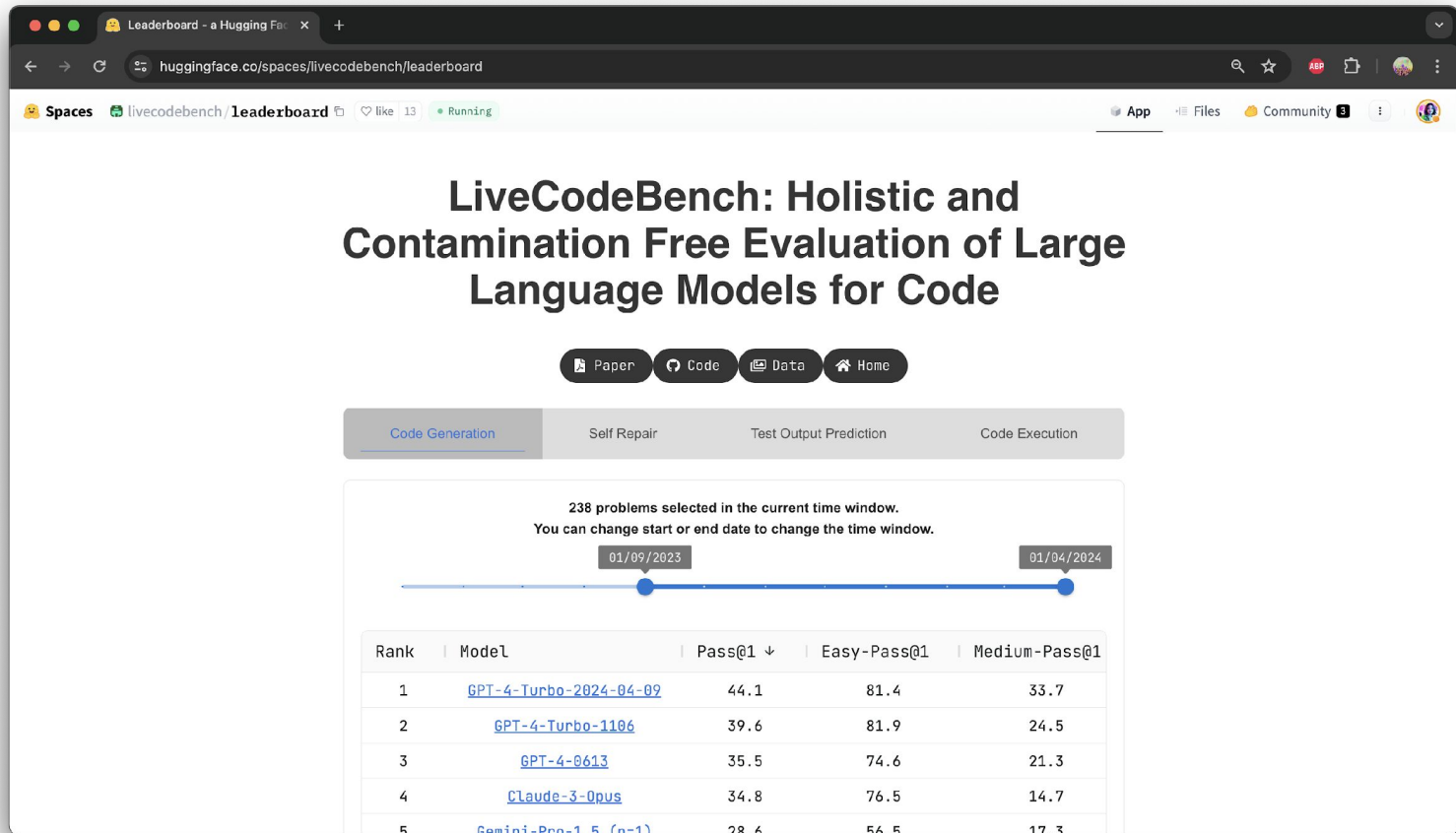
T	Model	Win Rate	humaneval-python	java	javascript	cpp
🔥 EXT	OpenCodeInterpreter-DS-33B	54.67	75.23	54.8	69.06	64.47
🔥	CodeQwen1.5-7B-Chat	53.83	87.2	61.04	70.31	67.85
🔥 EXT	CodeFuse-DeepSeek-33b	53.08	76.83	60.76	66.46	65.22
🔥 EXT	DeepSeek-Coder-33b-instruct	51.33	80.02	52.03	65.13	62.36
🔥 EXT	DeepSeek-Coder-7b-instruct	49.92	80.22	53.34	65.8	59.66
🔥 EXT	OpenCodeInterpreter-DS-6.7B	49.33	73.2	51.41	63.85	60.01
🔥	Phind-CodeLlama-34B-v2	48.5	71.95	54.06	65.34	59.59
🔥	Phind-CodeLlama-34B-v1	47.38	65.85	49.47	64.45	57.81
🔥	Phind-CodeLlama-34B-Python-v1	45.81	70.22	48.72	66.24	55.34
🟢	CodeQwen1.5-7B	44.92	50.79	42.15	50.07	48.35
🔥	CodeLlama-70b-instruct	43.33	75.6	47.2	57.76	48.45

Leaderboards

Potential Overfitting in HumanEval



Leaderboards



Thank you!